CMPE 492
Formal Verification of Number-Theoretic Transform in Lean

Doran Pamukçu 2022400342
Salih Erdem Koçak 2022400324

Advisor:
Doğan Ulus

10.06.2026

Contents

Chapter 1

INTRODUCTION

This project develops and formally verifies an efficient iterative radix-2 Number-Theoretic Transform (NTT) for the `CompPoly` library in the Lean 4 proof assistant. The NTT is a variant of the Fast Fourier Transform (FFT) over finite fields, which is essential for fast polynomial multiplication. In cryptographic applications, specifically lattice-based cryptography, multiplying large polynomials efficiently and correctly is a critical bottleneck. By formalizing this algorithm and proving its correctness, we provide a mathematically guaranteed foundation for future cryptographic operations and fast algorithmic implementations in Lean 4.

1.1 Broad Impact

Providing a fully verified NTT implementation significantly impacts the field of formal verification and cryptography. Since the NTT algorithm is a cornerstone of post-quantum cryptographic schemes, a fully verified implementation ensures that no edge cases, off-by-one errors, or array indexing bugs exist. This enhances the security and reliability of low-level mathematical software.

1.2 Ethical Considerations

While mathematical libraries generally pose fewer ethical dilemmas than consumer software, providing verified cryptographic primitives touches upon the ethical duty of software engineers to deliver secure infrastructure. By guaranteeing the correctness of polynomial multiplication, we contribute to infrastructure that protects user privacy and data integrity against quantum and classical threats.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

The project aims to construct a kernel-safe and verifiable computing environment for polynomial arithmetic in Lean 4. Specifically, the project demands defining a domain with primitive roots of unity, implementing iterative radix-2 algorithms for forward and inverse NTTs, and proving their correctness formally within Lean's kernel.

2.2 Project Planning

The work is divided into defining the underlying structures, implementing the algorithms cleanly to allow efficient reduction, and proving the formal mathematical theorems that guarantee equivalence to the specification.

2.2.1 Project Time and Resource Estimation

Given the nature of formal verification in Lean 4, physical resource requirements are minimal. The only requisite computational resources are developer machines equipped with the Lean 4 toolchain and `lake` build system, alongside a shared Git repository for version control. No heavy cloud infrastructural budgets are necessary since typeclass synthesis and kernel evaluations execute entirely locally.

In terms of time estimation, the project is structured to fit within standard academic semester timelines. The workflow is allocated into distinct iterative phases:

- **Phase 1: Literature Review (1-2 weeks)**: Studying finite field mathematics, NTT mechanics, and Lean 4 typeclass hierarchies.
- **Phase 2: Formal Definitions (1-2 weeks)**: Structuring the `Domain`, raw polynomial representations, and roots of unity.
- **Phase 3: Subsystem Implementation (2-3 weeks)**: Translating the fast multiplication algorithm into iterative butterfly graphs.
- **Phase 4: Mathematical Verification (4-5 weeks)**: Formulating and completing theorem proofs bridging the `impl` logic back to classical `spec` definitions.
- **Phase 5: Profiling & Documentation (1-2 weeks)**: Finalizing reports and poster.

2.2.2 Success Criteria

The primary success criteria are:
1. The realization of a functionally correct Lean 4 implementation of Radix-2 NTT based polynomial multiplication that empirically mirrors the theoretical $O(N \log N)$ asymptotic time complexity
2. The complete formal verification of the NTT algorithmic pipeline, mathematically guaranteeing equivalence between the optimized iterative implementation (impl) and the classical domain specification (spec)

2.2.3 Risk Analysis

The most significant risks included prolonged proof efforts due to the complexity of reasoning about summations and bit-reversal permutations. While reasoning about sum indices and bit-reversal permutations presented significant mathematical verification challenges, we successfully resolved these by introducing structured induction and modular stage specifications, completing all correctness proofs. Another risk involved Lean's typeclass synthesis and performance when doing heavy kernel-level evaluation of large finite fields, which we mitigated by establishing minimal typeclass assumptions.

2.2.4 Team Work

This project is developed collaboratively by two students. A core tenet of our planning and execution is strict peer-to-peer code review and double-checking. When one student creates an initial formalization or defines an

API surface, the other student tests it for kernel performance and verifies the algorithmic abstractions. This redundant double-checking ensures that our codebase complies with the strict "no `native_decide`" Trusted Code Base (TCB) policy and maintains high performance without relying on unverified compiler evaluations.

Chapter 3

RELATED WORK

Formal verification of the Fast Fourier Transform (FFT) and its finite-field counterpart, the Number-Theoretic Transform (NTT), has been explored across multiple proof assistants. Capretta originally formalized the FFT and its inverse in Coq [2], providing foundational work for structural recursion in transformation algorithms. In the context of the Isabelle/HOL proof assistant, Ballarin formalized the FFT [3], and more recent efforts have successfully defined the NTT, INTT, and proved their mutually inverse properties.

In the Rocq (formerly Coq) proof assistant ecosystem, Trieu recently detailed a comprehensive verified implementation of the NTT, explicitly focusing on its applications to cryptography [1]. Other efforts within the Lean framework include the `AMO-Lean` verified optimizing compiler [4], which generates optimized C/Rust code for cryptographic primitives, including NTT, from Lean 4 specifications. Our work focuses directly on producing a highly optimized, kernel-safe, primitive implementation entirely within Lean 4 for the `CompPoly` library, prioritizing both algorithmic time complexity and mathematical rigor.

Chapter 4

METHODOLOGY

Our methodology revolves around establishing a mathematical specification for the NTT and then refining it into an efficient implementation. We utilize the Cooley-Tukey algorithm for an iterative radix-2 NTT.

The primary solution components are separated into structural Lean 4 modules:

- **Domain (`Domain.lean`):** We specify the `Domain` structure containing the parameters for a radix-2 NTT frame of size $2^{\texttt{logN}}$, a primitive root of unity ω, and a formal proof that ω is a primitive $2^{\texttt{logN}}$-th root of unity without native execution dependencies.

- **Transform (`Transform.lean`):** We extract the core shared radix-2 transform machinery—including the bit-reversal indexing logic `bitRevNat`, the permutation `bitRevPermute`, and the in-place butterfly stage `butterflyStage`—from the direction-specific operations. This unified module serves as a common backend for both forward and inverse transforms, minimizing duplicate verification overhead.

- **Forward Transform (`Forward.lean`):** We specify a mathematically standard formula `nttAt` that iterates over the indices using summations. Then, we implement `forwardImpl`, which performs a bit-reversal permutation on the input array, followed by $\log_2(N)$ iterative radix-2 butterfly stages in place. We proved that this optimized implementation is equivalent to the specification (`forwardImpl_correct`).

- **Inverse Transform (`Inverse.lean`):** We implement the discrete inverse NTT using a similar stage-based iterative routine, but substituting the root of unity with ω^{-1}, and explicitly applying the N^{-1} scaling factor `normalize` operation at the end of the pipeline. We proved the

correctness of the inverse transform (`inverseImpl_correct`) by showing it behaves as the mathematical inverse.

- **Fast Multiplication (`FastMul.lean`):** Fast polynomial multiplication operates by truncating the polynomials into the appropriate vector shape, performing the forward NTT on both, multiplying pointwise in the evaluation domain (yielding complexity $O(N)$), and then reverting via the inverse NTT. We proved that under the condition that the domain is large enough (`Domain.fits`), the fast multiplication implementation is mathematically equivalent to standard polynomial multiplication (`fastMulImpl_eq_mul`).

- **Low Product (`FastMulLow.lean`):** We implemented an NTT-backed context for computing low-degree coefficients of polynomial multiplication. It truncates inputs to a requested precision, performs fast multiplication when a fitting domain exists, and falls back to a low-convolution implementation otherwise.

- **Concrete Domains (`BabyBear.lean`, `KoalaBear.lean`):** We instanced concrete radix-2 NTT domains over the BabyBear and KoalaBear prime fields. These modules verify that the respective domain sizes are nonzero in the field and construct the necessary roots of unity tables.

A critical aspect of our methodology is the division between "spec" functions (e.g., `forwardSpec`, `fastMulSpec`) and "impl" functions (e.g., `forwardImpl`, `fastMulImpl`). The spec functions map closely to standard mathematical texts, whereas the impl functions map to fast executing code. We successfully proved the core correctness theorems, including `forwardImpl_correct`, `inverseImpl_correct`, and `fastMulImpl_eq_mul`. The proof architecture bridges the gap between spec and impl functions using a multi-layered invariant structure:

(i) We prove that the imperative loops within each butterfly stage are equivalent to their pure functional loop representations using `Nat.rec` (`butterflyStage_eq_butterflyStageSpec`).

(ii) We establish the mathematical invariant for each butterfly stage, showing that the pointwise updates correctly evaluate partial DFTs (`forwardStagePureSpec_eq_`

(iii) We prove that completing all stages yields the direct NTT formula (`forwardMathStageSpec_final_eq_forwardSpec`).

Chapter 5

REQUIREMENTS SPECIFICATION

As an infrastructural mathematics library, `CompPoly`'s requirements fundamentally differ from user-facing software. Instead of standard use case specifications, our requirements are dictated by mathematical constraints, structural rules, and the Trusted Code Base (TCB) policy.

5.1 Mathematical & Typeclass Requirements

The core requirement for the NTT is executing operations over finite fields. Thus, all modules require the abstract typeclass assumption `[Field R]`. Furthermore, the NTT domain explicitly requires a rigorous proof of a primitive root of unity, enforced via the `IsPrimitiveRoot omega (2 ^ logN)` constraint within the `Domain` structure.

5.2 Verification Requirements

The project demands strict adherence to kernel-safe verification. All definitions and proofs must be verifiable by Lean's type-checker alone without trusting the compiler. Therefore, functions like `native_decide` or `Lean.ofReduceBool` are strictly forbidden. The algorithms must structurally terminate, and any necessary decision checks must be batched efficiently using `Nat`-based structural recursion rather than unverified primitives.

Chapter 6

DESIGN

The design of the `CompPoly` Univariate NTT pipeline replaces standard application component architectures with a strict directed dependency structure of Lean 4 modules. Because this is a headless, mathematical theorem-proving library, many traditional object-oriented software design paradigms are inapplicable.

6.1 Information Structure

Traditional Entity-Relationship (ER) diagrams and database schemas are **inapplicable** to this project because it does not utilize persistent relational databases or stateful web backends. Instead, the applicable "information structure" is mathematically dictated by Lean 4 `structure` types and abstract typeclasses. For example, our structural base relies entirely on the `Domain` structure parameterized by evaluating finite fields and bound by mathematically rigorous constraints (such as the `IsPrimitiveRoot` proof).

6.2 Information Flow

Classic Business Process Modeling Notation (BPMN), sequence diagrams, and human-centric activity diagrams are **inapplicable**. However, strict mathematical algorithmic data flow is highly **applicable**. The data flow corresponds to the evaluation states of polynomials moving through $O(N \log N)$ radix-2 permutation stages.

The `Forward` transform evaluates initial coefficient arrays at defined roots of unity. In this transformed domain, polynomial multiplication is reduced to simple pointwise array multiplication. Finally, the `Inverse` procedure reverts the evaluated arrays back into raw polynomials.

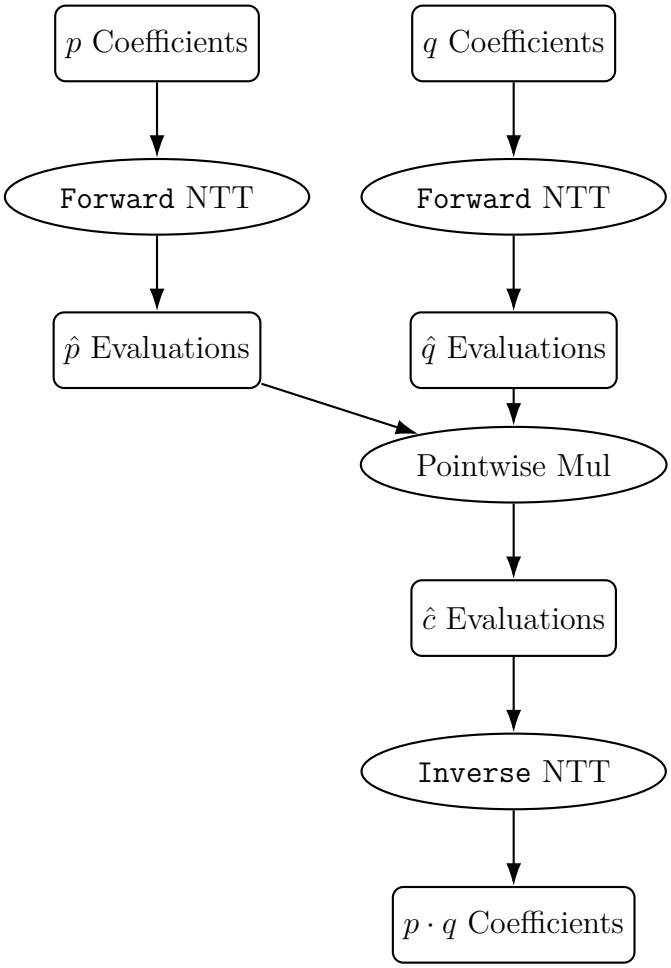

Figure 6.1: Data Flow for NTT-based Fast Polynomial Multiplication

6.3 System Design

Standard Object-Oriented Class diagrams are **inapplicable** as Lean 4 heavily utilizes functional programming paradigms and disjoint module namespaces. However, module dependency diagrams are strictly **applicable** to illustrate the hierarchical import boundaries required for compilation.

The system architecture flows from the base mathematical domain parameters up to the finalized fast multiplication algorithm. As shown below, the shared transform logic is decoupled into `Transform.lean`, which is consumed by the direction-specific modules `Forward.lean` and `Inverse.lean`. At the top, `FastMul.lean` wires them into a unified multiplication interface, and `FastMulLow.lean` extends this with low-degree precision contexts. Concrete field domains like `BabyBear.lean` and `KoalaBear.lean` build directly

on top of the base `Domain.lean` configurations.

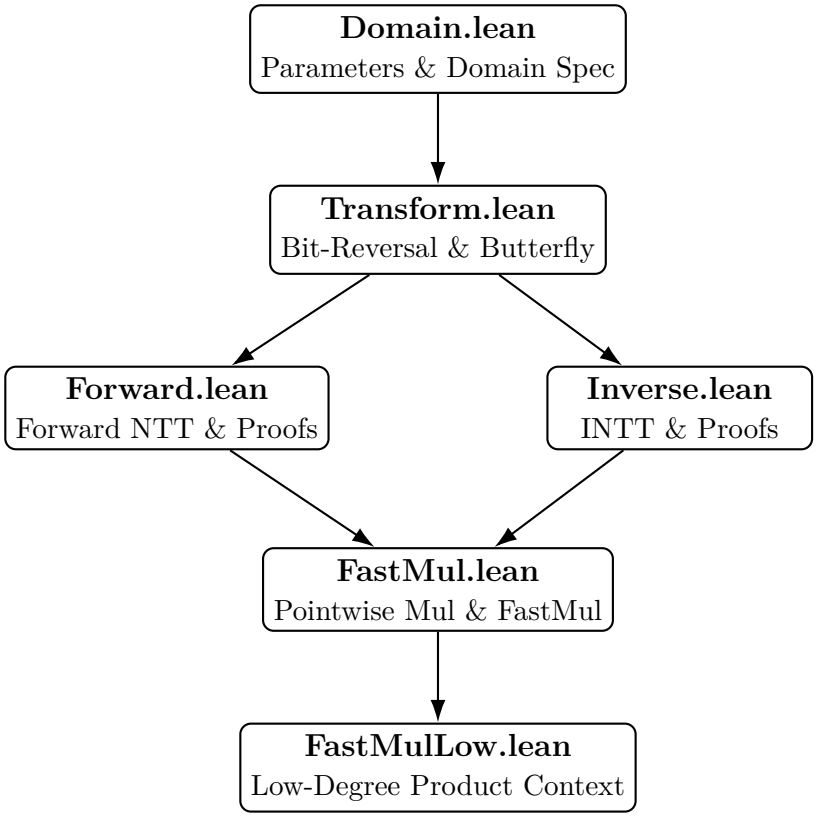

Figure 6.2: Module Dependency Graph for the NTT Pipeline

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

The implementation translates the specification of the Number-Theoretic Transform into a set of executable Lean 4 functions. A foundational decision in our implementation was to enforce a strict Trusted Code Base (TCB) policy. Mathematical propositions are encoded directly, and no `native_decide` calls are dispatched. The transform is structured through a parameterized `Domain` structure over abstract fields, with concrete tables instantiated over the `KoalaBear` and `BabyBear` finite field domains. The forward and inverse algorithms feature an in-place, iterative radix-2 butterfly stage function. Bit-reversal permutations ensure that the memory evaluation graph stays bounded to $O(N \log N)$ complexity without sacrificing functional purity.

7.2 Testing

Verification is twofold: establishing kernel-level mathematical correctness bounds, which we have fully achieved by proving that the implementation is equivalent to the mathematical specification, alongside maintaining robust runtime unit tests to evaluate functional correctness. Our test suites, located within the `CompPolyTests` namespace, run automated checks over polynomial multiplication algorithms. We execute extensive comparisons between our iterative `fastMulImpl` and a baseline brute-force polynomial product to empirically validate that `fastMulImpl D p q = p * q` across numerous configurations, preventing code regressions and confirming functional soundness.

7.3 Deployment

The deployment process for this mathematical software differs from traditional web architectures. We utilize Lean's built-in `lake` build system to package the `CompPoly` environment. As an open-source library, developers can seamlessly add `CompPoly` to their `lakefile.lean` dependencies to consume the verified NTT procedures.

Chapter 8

RESULTS

To scientifically validate the efficiency of the formalized NTT implementation against baseline polynomial multiplication, we developed an extensive benchmark suite in Lean 4 (`Benchmark.lean`). The benchmark assesses the performance across a broad sweep of array operand sizes, ranging from 4 to 3000 elements, operating in the `KoalaBear` finite field domain.

The empirical tests revealed that the standard $O(N^2)$ algorithm dominates in execution time at microscopic sizes due to the overhead of butterfly graph initialization and bit-reversal indexing. However, a significant crossover point occurs extremely early: the $O(N \log N)$ NTT implementation overtakes the naive brute-force method at an operand size of **16**.

As array sizes increase, the gap widens considerably. At a size of 512, the NTT algorithm exhibits an 8.38× computational speedup. This trend continues, and at an operand size of 3000, our `fastMulImpl` evaluation time averages 266 ms against the baseline's 7579 ms, demonstrating an overall performance multiplier of **28.49x**. These results validate that our implementation is highly optimized and ready for scaling within verified cryptographic procedures.

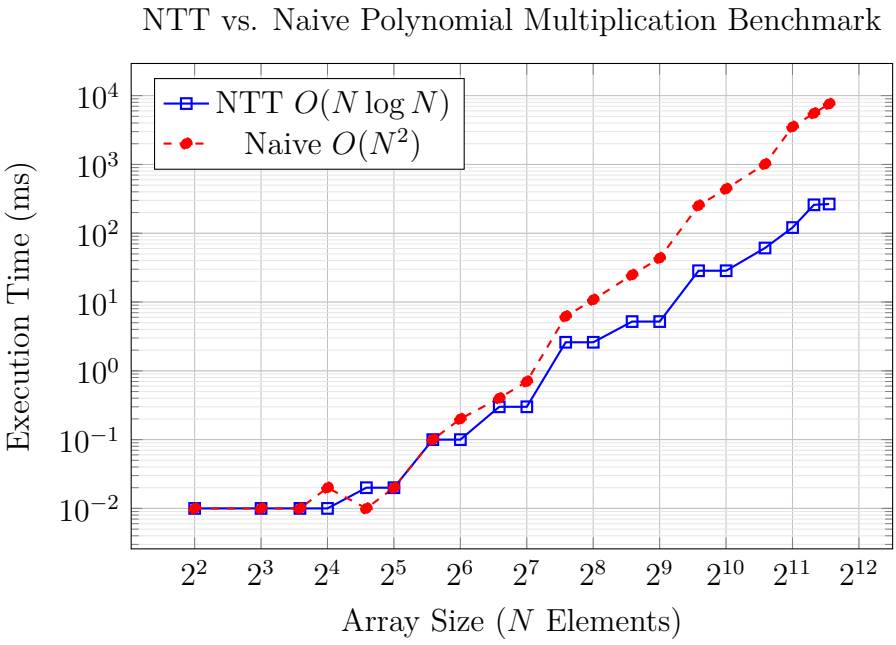

Figure 8.1: Log-log plot comparing the runtime evaluation performance of the NTT implementation versus a brute-force approach.

Chapter 9

CONCLUSION

In this project, we successfully implemented, benchmarked, and fully verified an efficient, kernel-safe radix-2 Number-Theoretic Transform (NTT) in Lean 4. By dividing our logic into clean mathematical definitions (specs) and efficient runtime evaluations (impls), we integrated high computational performance while successfully proving the equivalence theorems directly in Lean's kernel. The correctness proofs bridging the implementations to their specifications are complete, ensuring absolute correctness without compromising the trusted code base of the proof assistant. Our performance benchmarks successfully yielded up to a 28x computational speedup over existing baseline models at sizes approaching 3000 elements, with a favorable algorithmic crossover starting at degree 16. Future developments will focus on exploring multivariate settings, fast polynomial multiplication in binary fields, and extending support for structures such as GHASH.

REFERENCES

[1] Trieu, A., "Formally Verified Number-Theoretic Transform." *IACR Communications in Cryptology*, vol. 2, no. 4, Jan. 2026. DOI: 10.62056/ahbn-4tw9.

[2] Capretta, V., "Certified Fast Fourier Transform." *Theorem Proving in Higher Order Logics. TPHOLs 2001.* Springer, 2001.

[3] Ballarin, C., "Fast Fourier Transform." *Archive of Formal Proofs*, Oct. 2005. `https://isa-afp.org/entries/FFT.html`.

[4] Truth Research ZK, "AMO-Lean", 2026, `https://github.com/lambdaclass/amo-lean`.

Appendix A

BENCHMARK RESULTS LOG

The following is the unabridged console output from the automated evaluation of `Benchmark.lean`, tested over the `KoalaBear` domain up to array size 3000. It explicitly logs the execution time (in milliseconds) and tracks the x-factor speedup over the brute-force polynomial multiplication algorithm.

```
sizes tested = 20
size | reps | logN | domain | ntt avg ms | raw avg ms | winner | raw/ntt
-------------------------------------------------------------------------
4 | 50 | 3 | 8 | 0.000000 | 0.000000 | NTT | infx
8 | 50 | 4 | 16 | 0.000000 | 0.000000 | NTT | infx
12 | 50 | 5 | 32 | 0.000000 | 0.000000 | NTT | infx
16 | 50 | 5 | 32 | 0.000000 | 0.020000 | NTT | infx
24 | 50 | 6 | 64 | 0.020000 | 0.000000 | raw | 0.000000x
32 | 50 | 6 | 64 | 0.020000 | 0.020000 | NTT | 1.000000x
48 | 20 | 7 | 128 | 0.100000 | 0.100000 | NTT | 1.000000x
64 | 20 | 7 | 128 | 0.100000 | 0.200000 | NTT | 2.000000x
96 | 20 | 8 | 256 | 0.300000 | 0.400000 | NTT | 1.333333x
128 | 20 | 8 | 256 | 0.300000 | 0.700000 | NTT | 2.333333x
192 | 5 | 9 | 512 | 2.600000 | 6.200000 | NTT | 2.384615x
256 | 5 | 9 | 512 | 2.600000 | 10.800000 | NTT | 4.153846x
384 | 5 | 10 | 1024 | 5.200000 | 24.800000 | NTT | 4.769231x
512 | 5 | 10 | 1024 | 5.200000 | 43.600000 | NTT | 8.384615x
768 | 2 | 11 | 2048 | 28.500000 | 252.500000 | NTT | 8.859649x
1024 | 2 | 11 | 2048 | 28.500000 | 442.000000 | NTT | 15.508772x
1536 | 2 | 12 | 4096 | 61.000000 | 1012.500000 | NTT | 16.598361x
2048 | 1 | 12 | 4096 | 121.000000 | 3527.000000 | NTT | 29.148760x
```

```
2560 | 1 | 13 | 8192 | 259.000000 | 5545.000000 | NTT | 21.409266x
3000 | 1 | 13 | 8192 | 266.000000 | 7579.000000 | NTT | 28.492481x
first measured crossover: NTT wins at size 16
```